

MAEPIMS PROJECT

Team_Epi_7

2025-03-08

Loading the Required Datasets for the analysis

```
library(tidyverse)
library(readxl)
library(MMWRweek)
library(forecast)
library(prophet)
library(xgboost)
library(Metrics)
library(reshape2)
```

Loading the dataset

```
data <- read_excel("C:/Users/Agba Xchanger/Documents/Datasets/covid-19_dataset.xlsx")
```

Removing the columns without values

```
data <- subset(data, select = -c(extreme_poverty, hospital_beds_per_thousand, population))
```

Extracting the 2020 to 2022

```
data <- head(data, 1092)
```

Further removing the unwanted columns

```
data <- subset(data, select = -c(population_density, median_age, aged_65_older, aged_70_older, gdp_per_
                                cardiovasc_death_rate, diabetes_prevalence, female_smokers, male_smoke
                                handwashing_facilities, life_expectancy, human_development_index))
```

Add epidemiological week and year columns

```
data$epi_year <- MMWRweek(data$date)$MMWRyear
data$epi_week <- MMWRweek(data$date)$MMWRweek
```

Grouping the data by epidemiological week

```
weekly_data <- data %>%
  group_by(epi_year, epi_week) %>%
  summarise(
    total_cases = sum(new_cases, na.rm = TRUE),
    total_deaths = sum(new_deaths, na.rm = TRUE),
    avg_stringency = mean(stringency_index, na.rm = TRUE)
  )
```

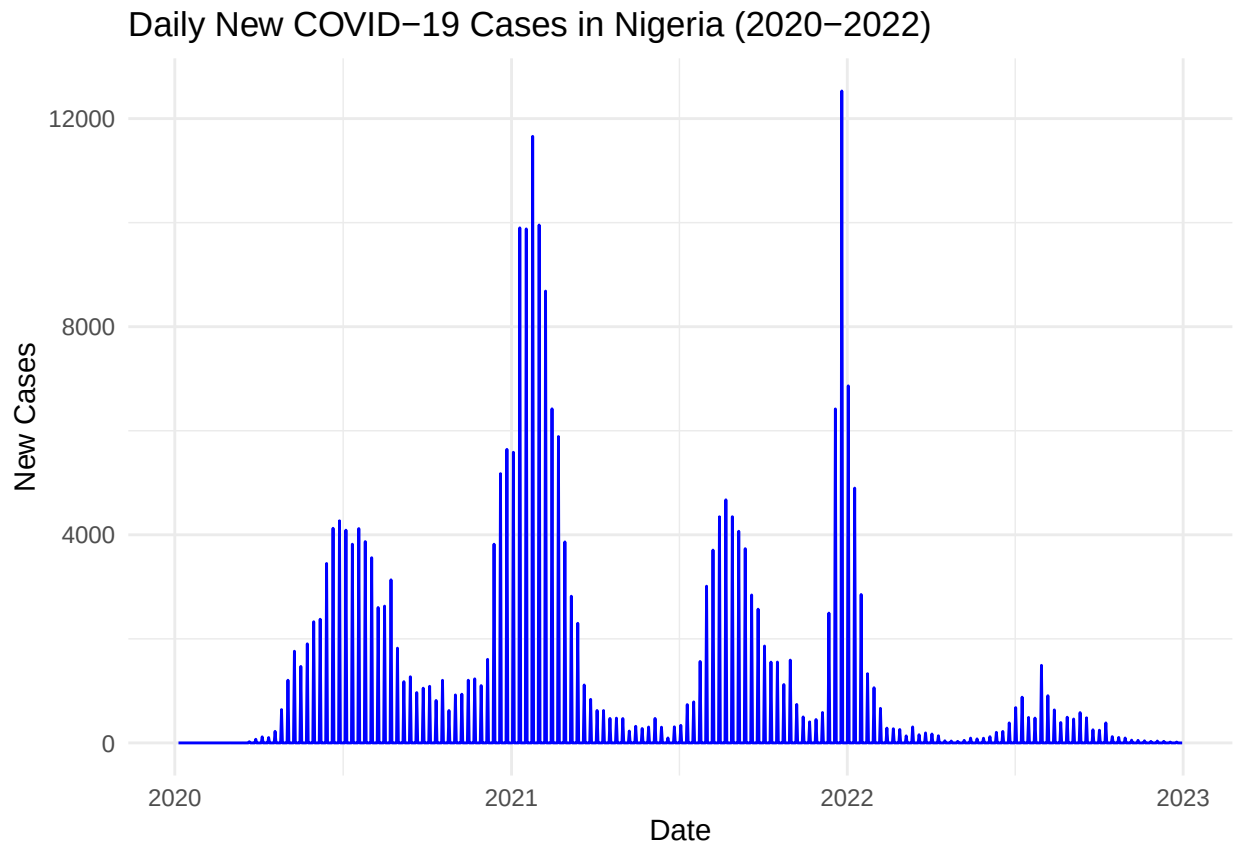
```
## `summarise()` has grouped output by 'epi_year'. You can override using the
## `.groups` argument.
```

```
#Viewing the
head(weekly_data)
```

```
## # A tibble: 6 x 5
## # Groups:   epi_year [1]
##   epi_year epi_week total_cases total_deaths avg_stringency
##   <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
## 1    2020         2         0         0         0
## 2    2020         3         0         0         0
## 3    2020         4         0         0        3.18
## 4    2020         5         0         0        7.15
## 5    2020         6         0         0        11.1
## 6    2020         7         0         0        11.1
```

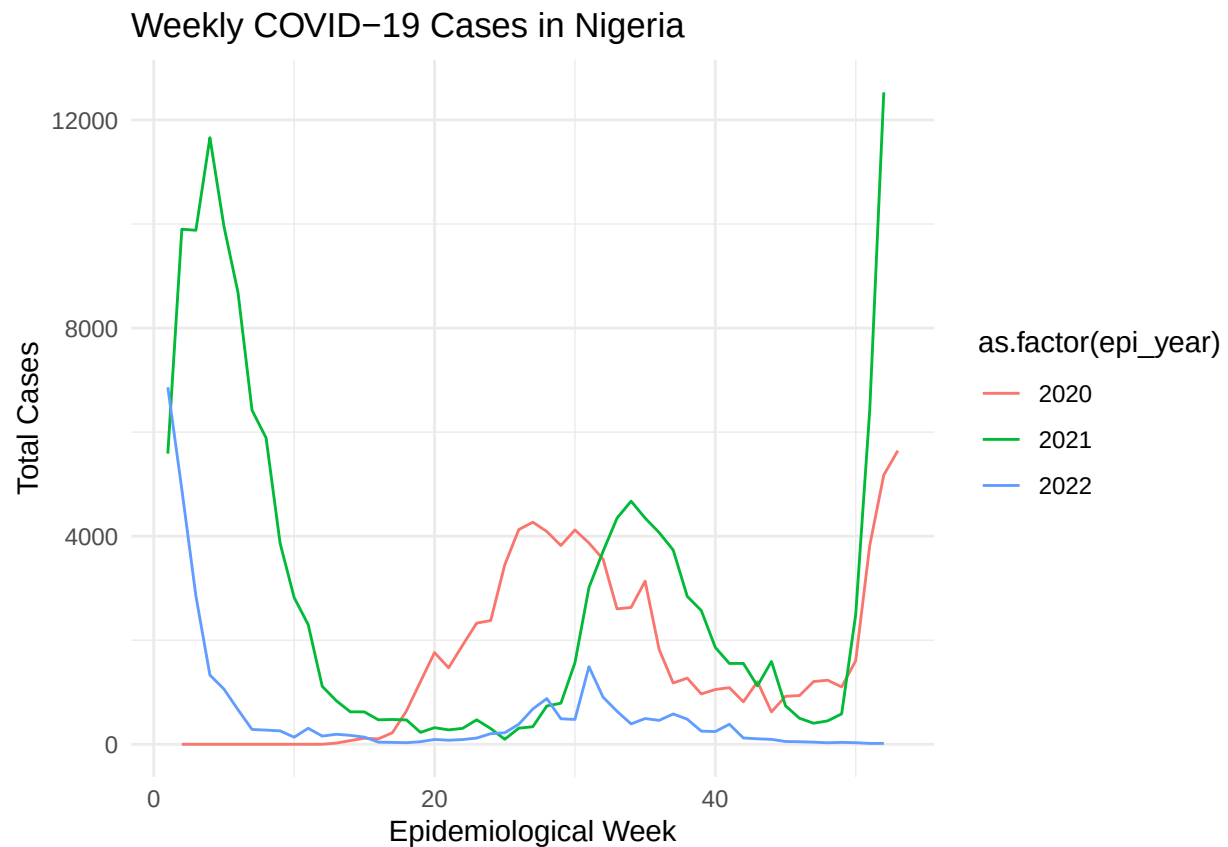
Plot Daily Cases Over Time

```
ggplot(data, aes(x = date, y = new_cases)) +
  geom_line(color = "blue") +
  labs(title = "Daily New COVID-19 Cases in Nigeria (2020-2022)", x = "Date", y = "New Cases") +
  theme_minimal()
```



Plot Daily Cases Over Time

```
ggplot(weekly_data, aes(x = epi_week, y = total_cases, group = epi_year, color = as.factor(epi_year))) +
  geom_line() +
  labs(title = "Weekly COVID-19 Cases in Nigeria", x = "Epidemiological Week", y = "Total Cases") +
  theme_minimal()
```



Correlation Analysis (Cases vs Stringency Index)

```
cor(data$new_cases, data$stringency_index, use = "complete.obs")
```

```
## [1] 0.1326733
```

MODEL SELECTION AND DEVELOPMENT

Training a Baseline Model (ARIMA)

Convert to time series

```
ts_data <- ts(data$new_cases, start = c(2020, 1), frequency = 365)
```

Fit ARIMA Model

```
model <- auto.arima(ts_data)
```

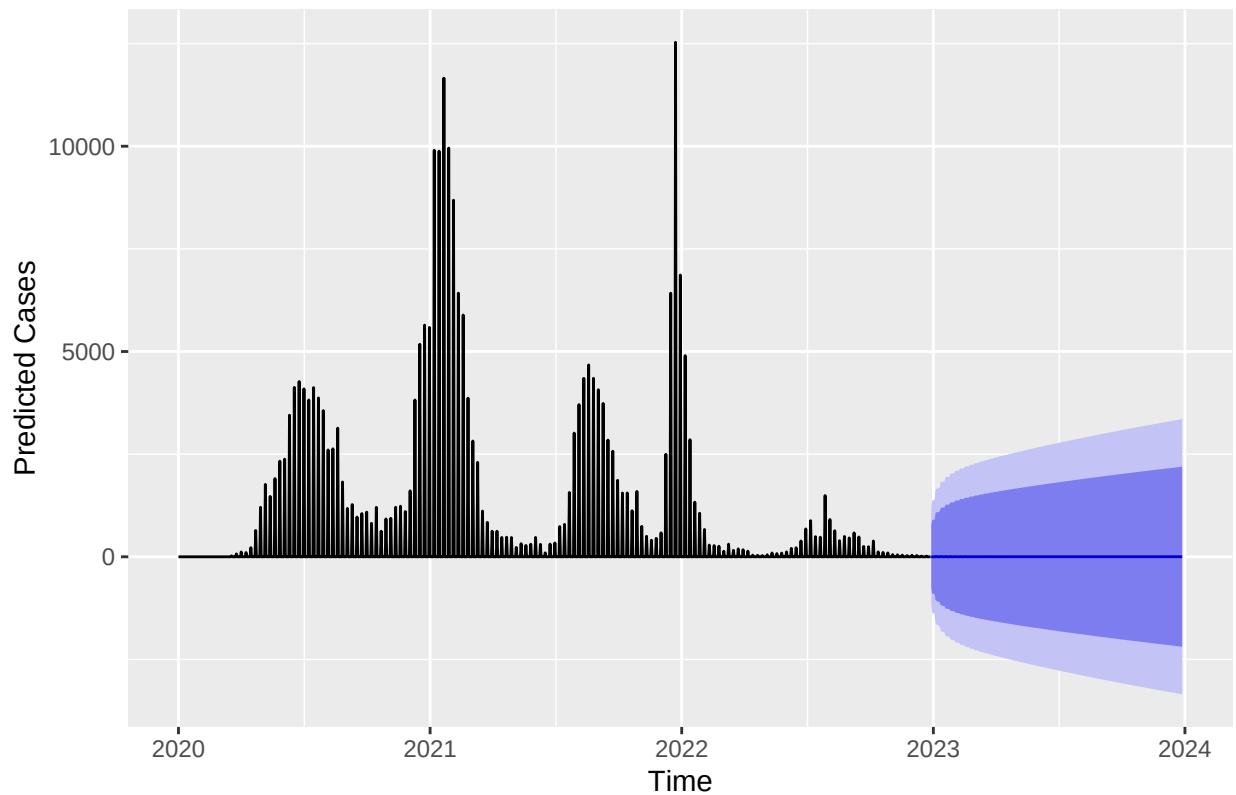
Forecast for 2022

```
forecast_2022 <- forecast(model, h = 365)
```

Plot Forecast

```
autoplot(forecast_2022) +  
  labs(title = "Forecasted COVID-19 Cases for 2022", x = "Time", y = "Predicted Cases")
```

Forecasted COVID-19 Cases for 2022



```
summary(model)
```

```
## Series: ts_data
## ARIMA(5,1,4)
##
## Coefficients:
##      ar1      ar2      ar3      ar4      ar5      ma1      ma2      ma3
##      0.0545 -0.7565 -0.3216 -0.3219 -0.6880 -1.6258  1.7258 -1.2614
## s.e.  0.0256  0.0204  0.0306  0.0200  0.0246  0.0316  0.0448  0.0382
##      ma4
##      0.5306
## s.e.  0.0258
##
## sigma^2 = 347495: log likelihood = -8508.41
## AIC=17036.83  AICc=17037.03  BIC=17086.77
##
## Training set error measures:
##              ME      RMSE      MAE MPE MAPE      MASE      ACF1
## Training set 0.01612445 586.7815 254.4102 NaN  Inf  0.4475476 -0.1345569
```

Fit the best SARIMA model using auto Arima

```
sarima_model <- auto.arima(ts_data, seasonal = TRUE)
summary(sarima_model)
```

```
## Series: ts_data
## ARIMA(5,1,4)
##
```

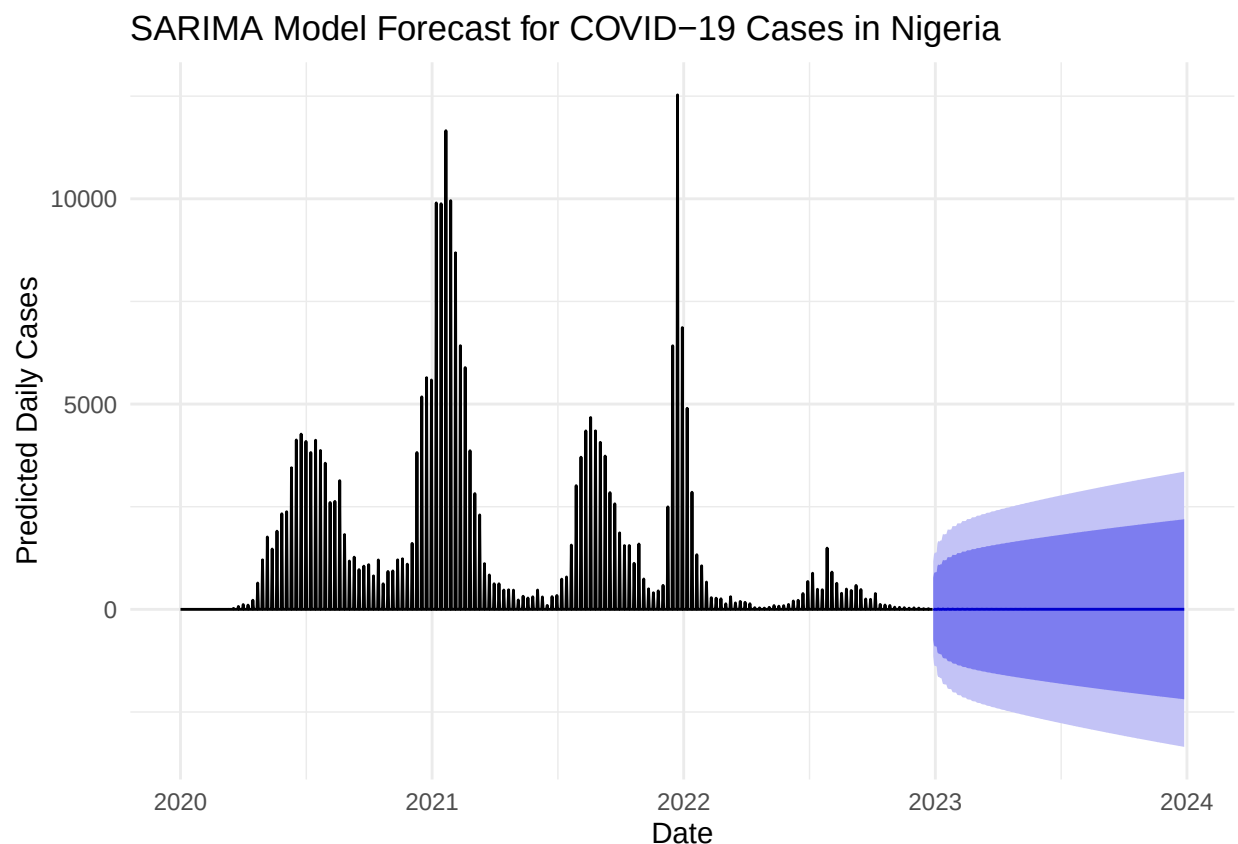
```
## Coefficients:
##      ar1      ar2      ar3      ar4      ar5      ma1      ma2      ma3
##      0.0545 -0.7565 -0.3216 -0.3219 -0.6880 -1.6258  1.7258 -1.2614
## s.e.  0.0256  0.0204  0.0306  0.0200  0.0246  0.0316  0.0448  0.0382
##      ma4
##      0.5306
## s.e.  0.0258
##
## sigma^2 = 347495: log likelihood = -8508.41
## AIC=17036.83 AICc=17037.03 BIC=17086.77
##
## Training set error measures:
##      ME      RMSE      MAE MPE MAPE      MASE      ACF1
## Training set 0.01612445 586.7815 254.4102 NaN  Inf  0.4475476 -0.1345569
```

Generate Forecast for 2022

```
forecast_sarima <- forecast(sarima_model, h = 365) # Forecasting for 1 year (365 days)
```

Plot the Forecast

```
autoplot(forecast_sarima) +
  labs(title = "SARIMA Model Forecast for COVID-19 Cases in Nigeria",
       x = "Date",
       y = "Predicted Daily Cases") +
  theme_minimal()
```

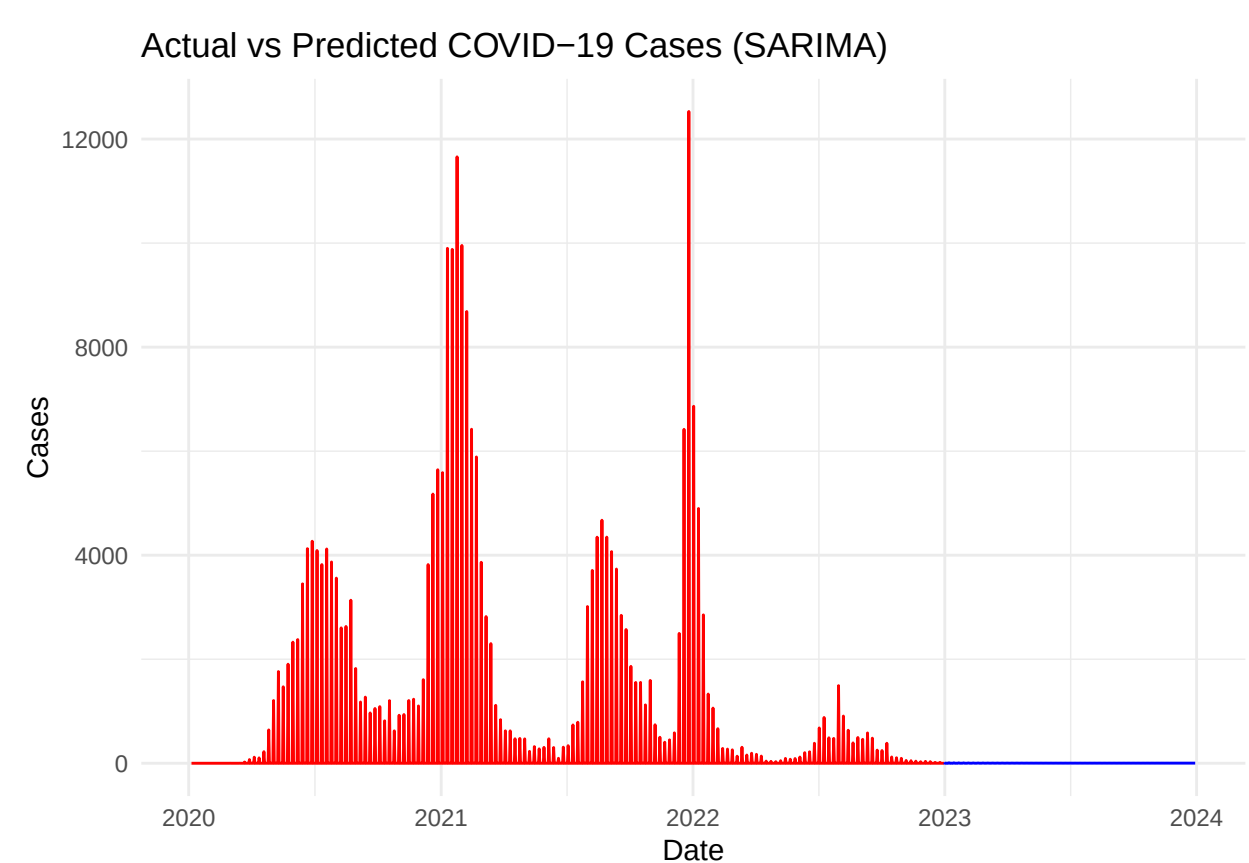


Convert forecast output to dataframe

```
forecast_df <- data.frame(
  date = seq(max(data$date) + 1, by = "day", length.out = 365),
  predicted_cases = as.numeric(forecast_sarima$mean)
)
```

Plot Actual vs Predicted Cases

```
ggplot() +
  geom_line(data = data, aes(x = date, y = new_cases), color = "red") + # Actual cases
  geom_line(data = forecast_df, aes(x = date, y = predicted_cases), color = "blue") +
  labs(title = "Actual vs Predicted COVID-19 Cases (SARIMA)",
       x = "Date",
       y = "Cases") +
  theme_minimal()
```



Prepare dataset for Prophet

```
prophet_data <- data.frame(ds = data$date, y = data$new_cases)
```

Fit Prophet model

```
prophet_model <- prophet(prophet_data)
```

Disabling daily seasonality. Run prophet with daily.seasonality=TRUE to override this.

Create future dataframe (365 days ahead)

```
future <- make_future_dataframe(prophet_model, periods = 365)
```

Generate predictions

```
forecast_prophet <- predict(prophet_model, future)
```

View forecast structure

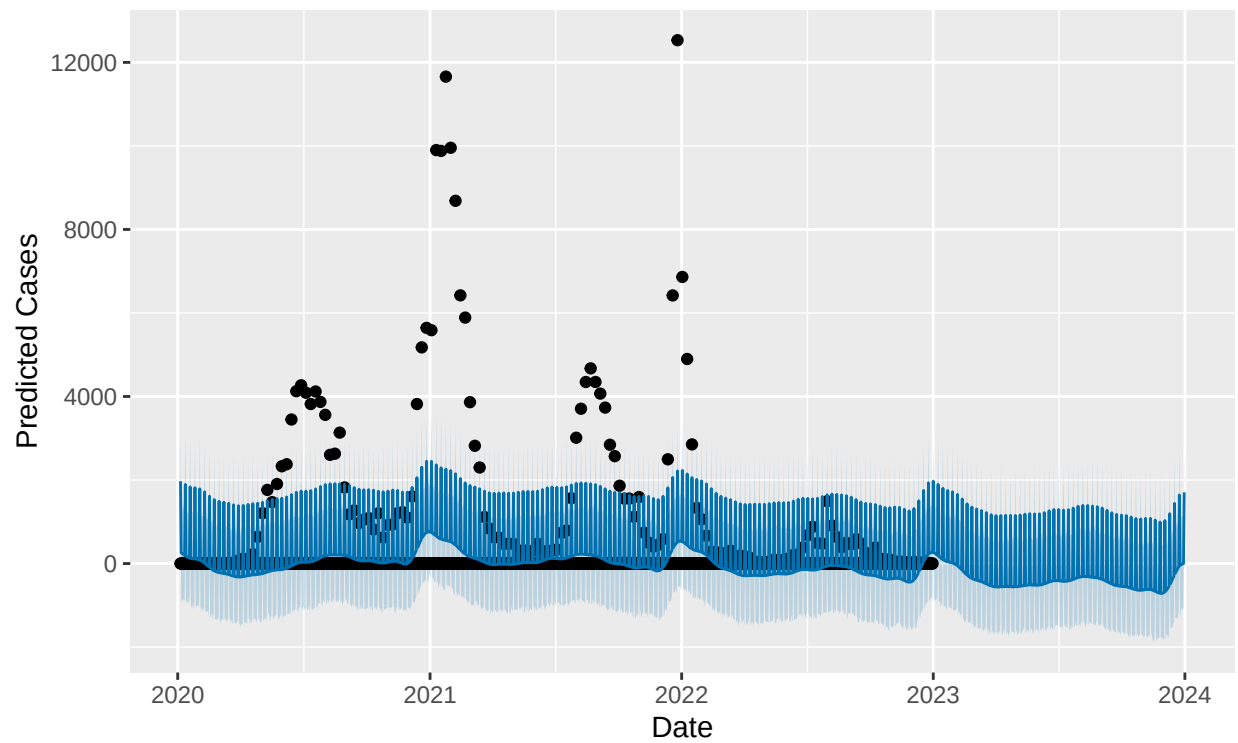
```
head(forecast_prophet)
```

```
##          ds          trend additive_terms additive_terms_lower additive_terms_upper
## 1 2020-01-05  2.398413      1969.5764      1969.5764      1969.5764
## 2 2020-01-06  4.002466       250.2767       250.2767       250.2767
## 3 2020-01-07  5.606519       237.7872       237.7872       237.7872
## 4 2020-01-08  7.210572       224.6159       224.6159       224.6159
## 5 2020-01-09  8.814625       211.0659       211.0659       211.0659
## 6 2020-01-10 10.418678       197.4222       197.4222       197.4222
##      weekly weekly_lower weekly_upper   yearly yearly_lower yearly_upper
## 1 1463.8050  1463.8050  1463.8050  505.7715   505.7715   505.7715
## 2 -243.9782 -243.9782 -243.9782  494.2548   494.2548   494.2548
## 3 -243.9641 -243.9641 -243.9641  481.7513   481.7513   481.7513
## 4 -243.9581 -243.9581 -243.9581  468.5740   468.5740   468.5740
## 5 -243.9593 -243.9593 -243.9593  455.0251   455.0251   455.0251
## 6 -243.9666 -243.9666 -243.9666  441.3887   441.3887   441.3887
##      multiplicative_terms multiplicative_terms_lower multiplicative_terms_upper
## 1                        0                        0                        0
## 2                        0                        0                        0
## 3                        0                        0                        0
## 4                        0                        0                        0
## 5                        0                        0                        0
## 6                        0                        0                        0
##      yhat_lower yhat_upper trend_lower trend_upper      yhat
## 1    849.1191   3043.228    2.398413    2.398413 1971.9748
## 2   -862.4905   1463.180    4.002466    4.002466  254.2792
## 3   -920.6470   1371.655    5.606519    5.606519  243.3937
## 4   -876.9853   1254.319    7.210572    7.210572  231.8265
## 5   -849.7524   1336.873    8.814625    8.814625  219.8805
## 6   -880.9283   1370.696   10.418678   10.418678  207.8408
```

Plotting the forecast

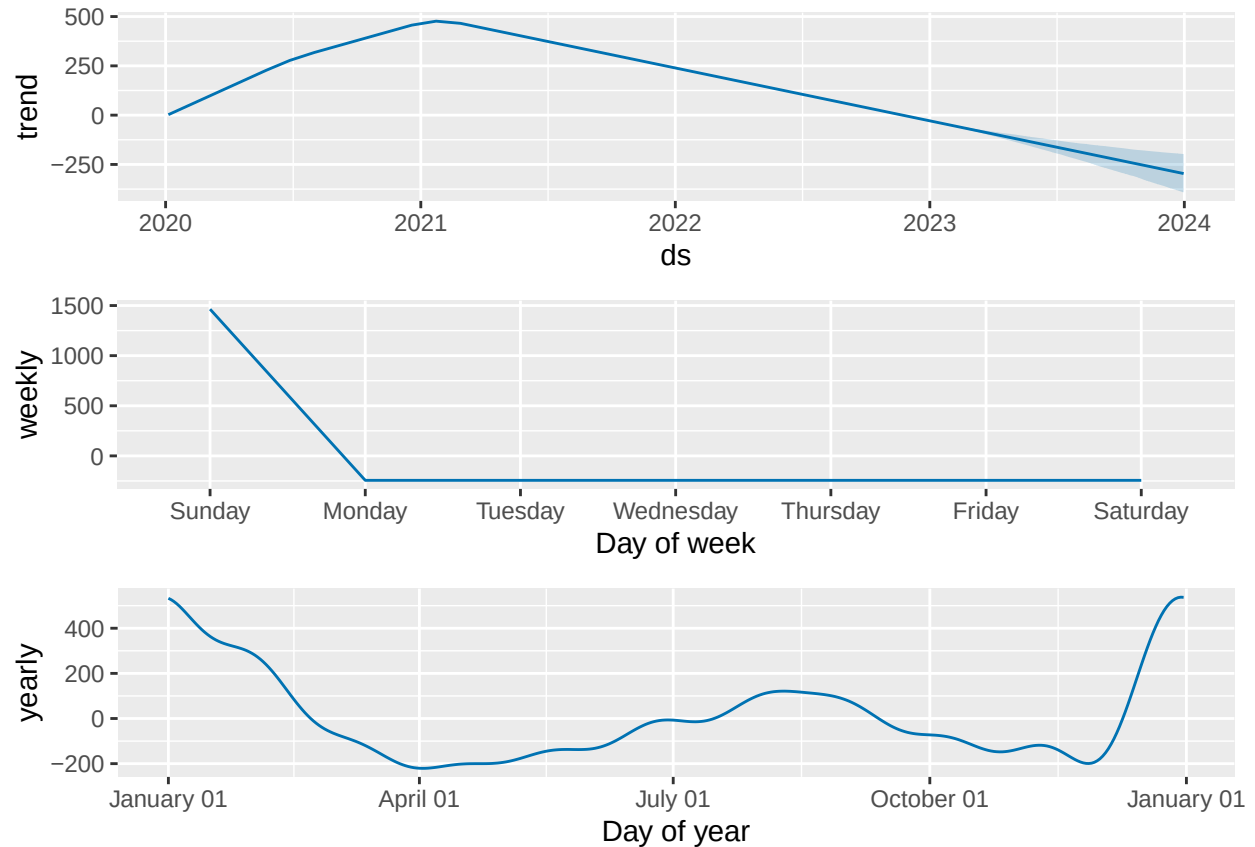
```
plot(prophet_model, forecast_prophet) +
  labs(title = "Prophet Forecast for COVID-19 Cases in Nigeria",
       x = "Date", y = "Predicted Cases")
```

Prophet Forecast for COVID-19 Cases in Nigeria



View Trend & Seasonality Components

```
prophet_plot_components(prophet_model, forecast_prophet)
```

Extract trend values

```
trend_data <- forecast_prophet[, c("ds", "trend")]
head(trend_data)
```

```
##           ds      trend
## 1 2020-01-05  2.398413
## 2 2020-01-06  4.002466
## 3 2020-01-07  5.606519
## 4 2020-01-08  7.210572
## 5 2020-01-09  8.814625
## 6 2020-01-10 10.418678
```

Extract weekly and yearly seasonal effects

```
seasonality_data <- forecast_prophet[, c("ds", "weekly", "yearly")]
head(seasonality_data)
```

```
##           ds    weekly    yearly
## 1 2020-01-05 1463.8050 505.7715
## 2 2020-01-06 -243.9782 494.2548
## 3 2020-01-07 -243.9641 481.7513
## 4 2020-01-08 -243.9581 468.5740
## 5 2020-01-09 -243.9593 455.0251
## 6 2020-01-10 -243.9666 441.3887
```

Extract weekly and yearly seasonal effects

```
seasonality_data <- forecast_prophet[, c("ds", "weekly", "yearly")]
head(seasonality_data)
```

```
##           ds    weekly   yearly
## 1 2020-01-05 1463.8050 505.7715
## 2 2020-01-06 -243.9782 494.2548
## 3 2020-01-07 -243.9641 481.7513
## 4 2020-01-08 -243.9581 468.5740
## 5 2020-01-09 -243.9593 455.0251
## 6 2020-01-10 -243.9666 441.3887
```

The full prophet model equation

```
forecast_prophet$prophet_model <- forecast_prophet$trend + forecast_prophet$weekly + forecast_prophet$yearly
head(forecast_prophet[, c("ds", "prophet_model")])
```

```
##           ds prophet_model
## 1 2020-01-05      1971.9748
## 2 2020-01-06       254.2792
## 3 2020-01-07       243.3937
## 4 2020-01-08       231.8265
## 5 2020-01-09       219.8805
## 6 2020-01-10       207.8408
```

XGBoost Model Prediction

Preparing data for XGBoost

Create lag features (e.g., last 7 days' cases)

```
data <- data %>%
  mutate(lag_1 = lag(new_cases, 1),
         lag_2 = lag(new_cases, 2),
         lag_3 = lag(new_cases, 3),
         lag_7 = lag(new_cases, 7)) %>%
  na.omit() # Remove rows with missing values
```

Define input (X) and output (y)

```
X <- as.matrix(data[, c("lag_1", "lag_2", "lag_3", "lag_7", "stringency_index")])
y <- as.numeric(data$new_cases)
```

Define split index

```
train_size <- round(0.8 * length(y))
```

Split data correctly

```
X_train <- X[1:train_size, , drop = FALSE]
y_train <- y[1:train_size]

X_test <- X[(train_size+1):length(y), , drop = FALSE]
y_test <- y[(train_size+1):length(y)]
```

Train XGBoost Model

```
xgb_model <- xgboost(data = X_train, label = y_train,
                    nrounds = 100, objective = "reg:squarederror")
```

```
## [1] train-rmse:933.226121
```

```
## [2] train-rmse:722.014290
## [3] train-rmse:568.129617
## [4] train-rmse:459.384022
## [5] train-rmse:379.806607
## [6] train-rmse:321.504622
## [7] train-rmse:270.037132
## [8] train-rmse:228.882925
## [9] train-rmse:196.469335
## [10] train-rmse:169.115951
## [11] train-rmse:146.801501
## [12] train-rmse:127.209093
## [13] train-rmse:111.512384
## [14] train-rmse:98.268161
## [15] train-rmse:86.676453
## [16] train-rmse:77.514913
## [17] train-rmse:70.115785
## [18] train-rmse:63.712362
## [19] train-rmse:58.221082
## [20] train-rmse:54.117957
## [21] train-rmse:49.507858
## [22] train-rmse:46.473540
## [23] train-rmse:43.722187
## [24] train-rmse:40.640694
## [25] train-rmse:38.550392
## [26] train-rmse:36.994120
## [27] train-rmse:35.762616
## [28] train-rmse:34.262563
## [29] train-rmse:31.548492
## [30] train-rmse:29.017117
## [31] train-rmse:28.051832
## [32] train-rmse:25.888290
## [33] train-rmse:24.372563
## [34] train-rmse:23.581498
## [35] train-rmse:22.077123
## [36] train-rmse:20.870408
## [37] train-rmse:20.153296
## [38] train-rmse:19.009896
## [39] train-rmse:18.499703
## [40] train-rmse:17.619586
## [41] train-rmse:16.821989
## [42] train-rmse:16.326758
## [43] train-rmse:15.185860
## [44] train-rmse:14.476431
## [45] train-rmse:13.413063
## [46] train-rmse:13.028966
## [47] train-rmse:12.406240
## [48] train-rmse:11.938401
## [49] train-rmse:11.212034
## [50] train-rmse:10.618496
## [51] train-rmse:10.335523
## [52] train-rmse:9.867982
## [53] train-rmse:9.265388
## [54] train-rmse:9.022940
## [55] train-rmse:8.648071
```

```
## [56] train-rmse:8.113147
## [57] train-rmse:7.642594
## [58] train-rmse:7.525380
## [59] train-rmse:7.167278
## [60] train-rmse:6.871355
## [61] train-rmse:6.736702
## [62] train-rmse:6.449662
## [63] train-rmse:6.328822
## [64] train-rmse:6.100045
## [65] train-rmse:5.982125
## [66] train-rmse:5.739188
## [67] train-rmse:5.493486
## [68] train-rmse:5.117481
## [69] train-rmse:4.996357
## [70] train-rmse:4.933420
## [71] train-rmse:4.832331
## [72] train-rmse:4.743559
## [73] train-rmse:4.604878
## [74] train-rmse:4.409807
## [75] train-rmse:4.314458
## [76] train-rmse:4.148298
## [77] train-rmse:3.963959
## [78] train-rmse:3.913171
## [79] train-rmse:3.773105
## [80] train-rmse:3.728618
## [81] train-rmse:3.558853
## [82] train-rmse:3.424620
## [83] train-rmse:3.287741
## [84] train-rmse:3.264342
## [85] train-rmse:3.187499
## [86] train-rmse:3.086496
## [87] train-rmse:2.963226
## [88] train-rmse:2.945146
## [89] train-rmse:2.895087
## [90] train-rmse:2.790428
## [91] train-rmse:2.706219
## [92] train-rmse:2.620471
## [93] train-rmse:2.552200
## [94] train-rmse:2.475849
## [95] train-rmse:2.463592
## [96] train-rmse:2.364525
## [97] train-rmse:2.329700
## [98] train-rmse:2.312012
## [99] train-rmse:2.220438
## [100] train-rmse:2.142354
```

Make Predictions

```
predictions_xgb <- predict(xgb_model, X_test)
```

Evaluating the Model Performance

Calculate RMSE and MAE

```
rmse_xgb <- rmse(y_test, predictions_xgb)
mae_xgb <- mae(y_test, predictions_xgb)
```

```
print(paste("XGBoost RMSE:", rmse_xgb))
```

```
## [1] "XGBoost RMSE: 139.732512468481"
```

```
print(paste("XGBoost MAE:", mae_xgb))
```

```
## [1] "XGBoost MAE: 29.223295662638"
```

Plotting actual vs predicted cases

```
ggplot() +
```

```
  # Actual cases in red
```

```
  geom_line(aes(x = data$date[(train_size+1):nrow(data)],  
                y = y_test, color = "Actual Cases")) +
```

```
  # Predicted cases in blue
```

```
  geom_line(aes(x = data$date[(train_size+1):nrow(data)],  
                y = predictions_xgb, color = "Predicted Cases")) +
```

```
  # Labels and theme
```

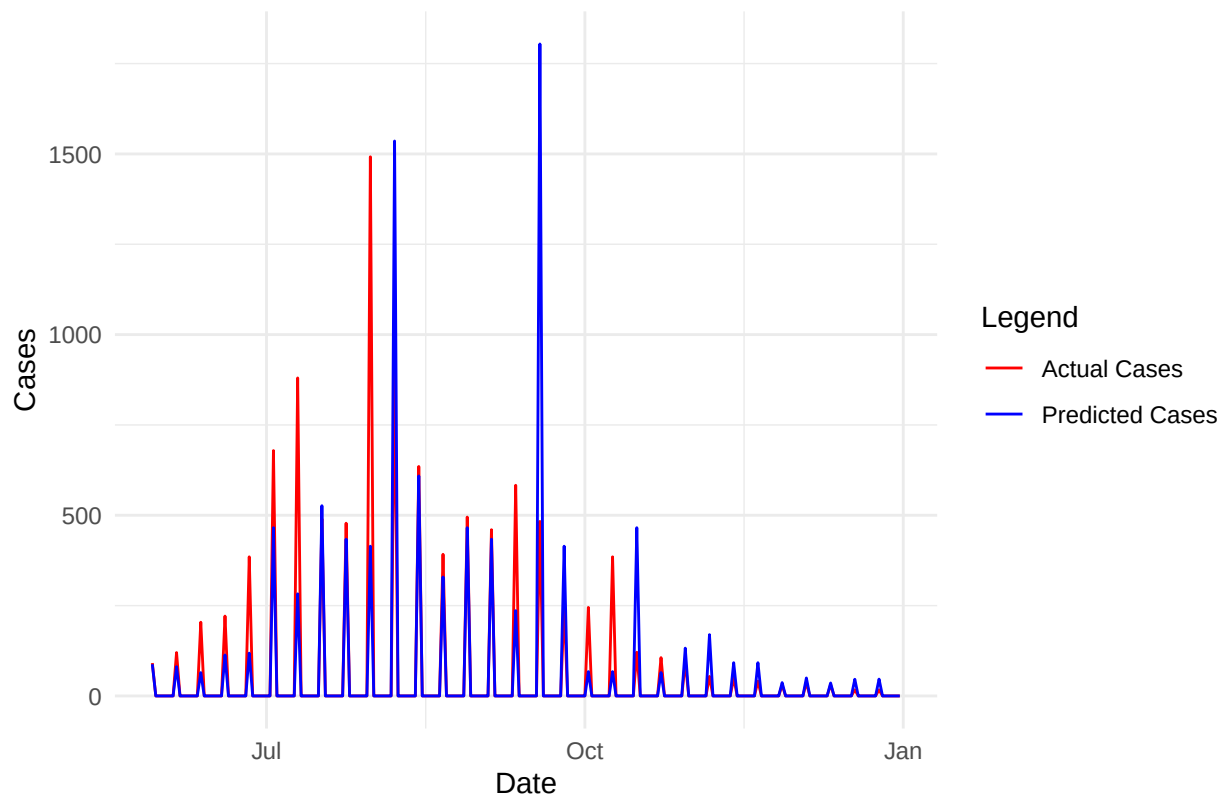
```
  labs(title = "XGBoost: Actual vs Predicted COVID-19 Cases",
```

```
        x = "Date", y = "Cases", color = "Legend") + # Add legend title
```

```
  scale_color_manual(values = c("Actual Cases" = "red", "Predicted Cases" = "blue")) + # Define colors
```

```
  theme_minimal()
```

XGBoost: Actual vs Predicted COVID-19 Cases



Comparison of SARIMA, PROPHET, AND XGBOOST MODEL

Extract SARIMA predictions for test set

```
sarima_forecast <- forecast(sarima_model, h = length(y_test))$mean
```

Calculate Errors

```
mae_sarima <- mae(y_test, sarima_forecast)
rmse_sarima <- rmse(y_test, sarima_forecast)
mape_sarima <- mape(y_test, sarima_forecast) * 100 # Convert to percentage

print(paste("SARIMA - MAE:", mae_sarima, "RMSE:", rmse_sarima, "MAPE:", mape_sarima))
```

```
## [1] "SARIMA - MAE: 49.417033661745 RMSE: 177.499476203696 MAPE: Inf"
```

Extract Prophet predictions for test set

```
prophet_forecast <- forecast_prophet$yhat[(train_size+1):nrow(data)]
```

Calculate Errors

```
mae_prophet <- mae(y_test, prophet_forecast)
rmse_prophet <- rmse(y_test, prophet_forecast)
mape_prophet <- mape(y_test, prophet_forecast) * 100 # Convert to percentage

print(paste("Prophet - MAE:", mae_prophet, "RMSE:", rmse_prophet, "MAPE:", mape_prophet))
```

```
## [1] "Prophet - MAE: 351.557995452796 RMSE: 503.59760161403 MAPE: Inf"
```

Calculate Errors for XGBoost

```
mae_xgb <- mae(y_test, predictions_xgb)
rmse_xgb <- rmse(y_test, predictions_xgb)
mape_xgb <- mape(y_test, predictions_xgb) * 100 # Convert to percentage

print(paste("XGBoost - MAE:", mae_xgb, "RMSE:", rmse_xgb, "MAPE:", mape_xgb))
```

```
## [1] "XGBoost - MAE: 29.223295662638 RMSE: 139.732512468481 MAPE: Inf"
```

Create a summary dataframe

```
model_comparison <- data.frame(
  Model = c("SARIMA", "Prophet", "XGBoost"),
  MAE = c(mae_sarima, mae_prophet, mae_xgb),
  RMSE = c(rmse_sarima, rmse_prophet, rmse_xgb),
  MAPE = c(mape_sarima, mape_prophet, mape_xgb)
)

print(model_comparison)
```

```
##      Model      MAE      RMSE MAPE
## 1 SARIMA  49.41703 177.4995  Inf
## 2 Prophet 351.55800 503.5976  Inf
## 3 XGBoost  29.22330 139.7325  Inf
```

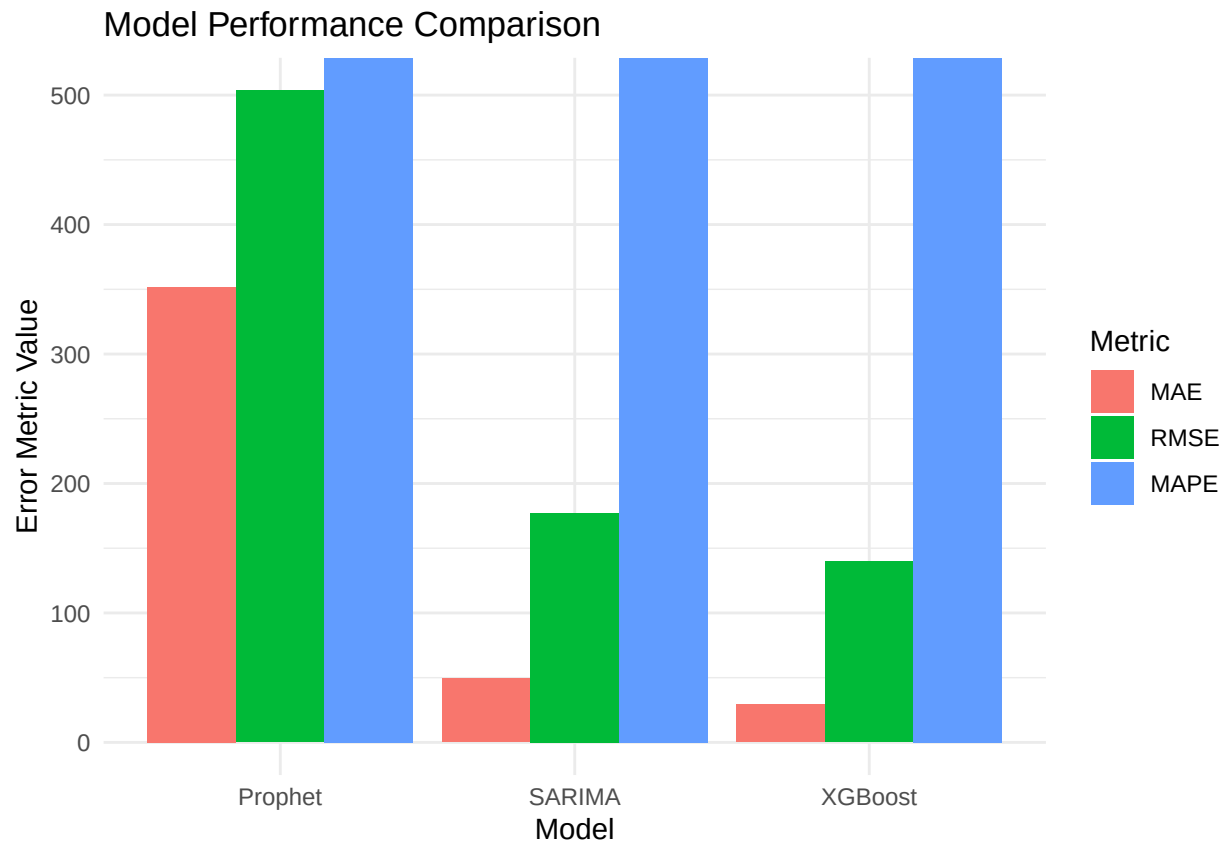
Visualizing Model Comparisons with a Bar Chart

Prepare Data for Plotting

```
model_comparison_melted <- melt(model_comparison, id = "Model")
```

Create Bar Chart

```
ggplot(model_comparison_melted, aes(x = Model, y = value, fill = variable)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Model Performance Comparison",
       x = "Model",
       y = "Error Metric Value",
       fill = "Metric") +
  theme_minimal()
```



Fine tuning the best model (XGBoost)

```
xgb_tuned <- xgboost(data = X_train, label = y_train, nrounds = 200, max_depth = 6, eta = 0.05)
```

```
## [1] train-rmse:1190.138893
## [2] train-rmse:1140.451794
## [3] train-rmse:1093.353250
## [4] train-rmse:1048.727103
## [5] train-rmse:1006.421486
## [6] train-rmse:966.339557
## [7] train-rmse:928.408032
## [8] train-rmse:892.537915
## [9] train-rmse:858.621280
## [10] train-rmse:826.606995
## [11] train-rmse:795.335278
## [12] train-rmse:766.091822
## [13] train-rmse:737.789954
## [14] train-rmse:710.895231
## [15] train-rmse:685.964046
```

```
## [16] train-rmse:661.553262
## [17] train-rmse:639.173639
## [18] train-rmse:617.093416
## [19] train-rmse:596.165468
## [20] train-rmse:576.377916
## [21] train-rmse:558.219266
## [22] train-rmse:540.291242
## [23] train-rmse:522.812295
## [24] train-rmse:506.491906
## [25] train-rmse:490.540828
## [26] train-rmse:475.700410
## [27] train-rmse:460.509869
## [28] train-rmse:445.939215
## [29] train-rmse:431.934320
## [30] train-rmse:419.930103
## [31] train-rmse:407.055230
## [32] train-rmse:394.785914
## [33] train-rmse:384.352191
## [34] train-rmse:373.896581
## [35] train-rmse:364.391075
## [36] train-rmse:353.784104
## [37] train-rmse:343.649319
## [38] train-rmse:334.910565
## [39] train-rmse:325.108646
## [40] train-rmse:315.991154
## [41] train-rmse:307.219378
## [42] train-rmse:298.669034
## [43] train-rmse:290.570613
## [44] train-rmse:282.680654
## [45] train-rmse:275.185944
## [46] train-rmse:267.914292
## [47] train-rmse:260.927898
## [48] train-rmse:254.223904
## [49] train-rmse:247.747487
## [50] train-rmse:241.476933
## [51] train-rmse:235.291865
## [52] train-rmse:229.442161
## [53] train-rmse:223.631651
## [54] train-rmse:218.168936
## [55] train-rmse:212.756357
## [56] train-rmse:207.455737
## [57] train-rmse:202.488822
## [58] train-rmse:197.479084
## [59] train-rmse:192.768096
## [60] train-rmse:188.324715
## [61] train-rmse:183.949925
## [62] train-rmse:179.723566
## [63] train-rmse:175.584503
## [64] train-rmse:171.623035
## [65] train-rmse:167.742030
## [66] train-rmse:163.980336
## [67] train-rmse:160.340836
## [68] train-rmse:156.664297
## [69] train-rmse:153.249936
```



```
## [70] train-rmse:149.925155
## [71] train-rmse:146.647948
## [72] train-rmse:143.513443
## [73] train-rmse:140.486758
## [74] train-rmse:137.434991
## [75] train-rmse:134.592658
## [76] train-rmse:131.817064
## [77] train-rmse:129.094051
## [78] train-rmse:126.275970
## [79] train-rmse:123.717578
## [80] train-rmse:121.122312
## [81] train-rmse:118.603975
## [82] train-rmse:116.100317
## [83] train-rmse:113.754648
## [84] train-rmse:111.474036
## [85] train-rmse:109.231830
## [86] train-rmse:107.001003
## [87] train-rmse:104.885177
## [88] train-rmse:102.700992
## [89] train-rmse:100.542246
## [90] train-rmse:98.704516
## [91] train-rmse:96.770529
## [92] train-rmse:95.021993
## [93] train-rmse:93.165264
## [94] train-rmse:91.389803
## [95] train-rmse:89.660930
## [96] train-rmse:87.961181
## [97] train-rmse:86.363017
## [98] train-rmse:84.748279
## [99] train-rmse:83.267343
## [100] train-rmse:81.614416
## [101] train-rmse:80.137527
## [102] train-rmse:78.721835
## [103] train-rmse:77.184917
## [104] train-rmse:75.863083
## [105] train-rmse:74.540043
## [106] train-rmse:73.140018
## [107] train-rmse:71.736668
## [108] train-rmse:70.541873
## [109] train-rmse:69.380698
## [110] train-rmse:68.251444
## [111] train-rmse:67.112038
## [112] train-rmse:66.057539
## [113] train-rmse:65.026777
## [114] train-rmse:63.964802
## [115] train-rmse:62.964615
## [116] train-rmse:62.000597
## [117] train-rmse:61.079063
## [118] train-rmse:60.129492
## [119] train-rmse:59.209548
## [120] train-rmse:58.307511
## [121] train-rmse:57.448253
## [122] train-rmse:56.607944
## [123] train-rmse:55.782771
```

```
## [124] train-rmse:54.987575
## [125] train-rmse:54.223805
## [126] train-rmse:53.490488
## [127] train-rmse:52.696448
## [128] train-rmse:51.960639
## [129] train-rmse:51.253811
## [130] train-rmse:50.385208
## [131] train-rmse:49.631670
## [132] train-rmse:49.048638
## [133] train-rmse:48.380424
## [134] train-rmse:47.673624
## [135] train-rmse:47.053606
## [136] train-rmse:46.457253
## [137] train-rmse:45.843376
## [138] train-rmse:45.308906
## [139] train-rmse:44.667498
## [140] train-rmse:44.095784
## [141] train-rmse:43.532714
## [142] train-rmse:43.029257
## [143] train-rmse:42.483630
## [144] train-rmse:41.926600
## [145] train-rmse:41.448321
## [146] train-rmse:40.952768
## [147] train-rmse:40.454057
## [148] train-rmse:40.009755
## [149] train-rmse:39.512254
## [150] train-rmse:39.052023
## [151] train-rmse:38.622535
## [152] train-rmse:38.194078
## [153] train-rmse:37.713741
## [154] train-rmse:37.323154
## [155] train-rmse:36.927119
## [156] train-rmse:36.528201
## [157] train-rmse:36.157870
## [158] train-rmse:35.799312
## [159] train-rmse:35.381041
## [160] train-rmse:35.040543
## [161] train-rmse:34.722325
## [162] train-rmse:34.396010
## [163] train-rmse:34.106732
## [164] train-rmse:33.795301
## [165] train-rmse:33.513288
## [166] train-rmse:33.229166
## [167] train-rmse:32.944188
## [168] train-rmse:32.677629
## [169] train-rmse:32.395482
## [170] train-rmse:32.154650
## [171] train-rmse:31.822317
## [172] train-rmse:31.598738
## [173] train-rmse:31.378428
## [174] train-rmse:31.089024
## [175] train-rmse:30.805327
## [176] train-rmse:30.651851
## [177] train-rmse:30.348428
```

```
## [178]    train-rmse:30.061396
## [179]    train-rmse:29.918954
## [180]    train-rmse:29.646734
## [181]    train-rmse:29.253178
## [182]    train-rmse:28.995240
## [183]    train-rmse:28.872454
## [184]    train-rmse:28.671521
## [185]    train-rmse:28.495703
## [186]    train-rmse:28.164763
## [187]    train-rmse:28.055666
## [188]    train-rmse:27.890889
## [189]    train-rmse:27.787308
## [190]    train-rmse:27.549063
## [191]    train-rmse:27.391144
## [192]    train-rmse:27.240231
## [193]    train-rmse:26.938722
## [194]    train-rmse:26.849732
## [195]    train-rmse:26.765623
## [196]    train-rmse:26.624791
## [197]    train-rmse:26.426905
## [198]    train-rmse:26.295630
## [199]    train-rmse:26.218254
## [200]    train-rmse:25.992592
```

Evaluating the fine tuned model Calculate Errors for XGBoost

```
mae_xgb <- mae(y_test, predictions_xgb)
rmse_xgb <- rmse(y_test, predictions_xgb)
mape_xgb <- mape(y_test, predictions_xgb) * 100 # Convert to percentage
```

Print Final Evaluation

```
print(paste("Final XGBoost Model - MAE:", mae_xgb, "RMSE:", rmse_xgb, "MAPE:", mape_xgb))
```

```
## [1] "Final XGBoost Model - MAE: 29.223295662638 RMSE: 139.732512468481 MAPE: Inf"
```

Generating final forecast for 2022 Retrain XGBoost on the full dataset

```
xgb_final <- xgboost(data = X, label = y, nrounds = 200, max_depth = 6, eta = 0.05, objective = "reg:sqrerror")
```

```
## [1]    train-rmse:1067.289355
## [2]    train-rmse:1022.789961
## [3]    train-rmse:980.645919
## [4]    train-rmse:940.591046
## [5]    train-rmse:902.677101
## [6]    train-rmse:866.733595
## [7]    train-rmse:832.723214
## [8]    train-rmse:800.600783
## [9]    train-rmse:770.182171
## [10]    train-rmse:741.468625
## [11]    train-rmse:713.496947
## [12]    train-rmse:687.331666
## [13]    train-rmse:662.005954
## [14]    train-rmse:637.905749
## [15]    train-rmse:615.617089
## [16]    train-rmse:593.742168
## [17]    train-rmse:573.687792
```

```
## [18] train-rmse:553.927155
## [19] train-rmse:535.215514
## [20] train-rmse:517.703142
## [21] train-rmse:500.754122
## [22] train-rmse:484.394772
## [23] train-rmse:468.856142
## [24] train-rmse:454.106086
## [25] train-rmse:439.855534
## [26] train-rmse:426.498140
## [27] train-rmse:412.916197
## [28] train-rmse:400.607258
## [29] train-rmse:388.043126
## [30] train-rmse:377.286869
## [31] train-rmse:365.721211
## [32] train-rmse:356.033462
## [33] train-rmse:345.425048
## [34] train-rmse:336.059191
## [35] train-rmse:327.574130
## [36] train-rmse:317.833170
## [37] train-rmse:308.618140
## [38] train-rmse:299.562969
## [39] train-rmse:292.530529
## [40] train-rmse:284.195055
## [41] train-rmse:276.313908
## [42] train-rmse:268.598458
## [43] train-rmse:261.357366
## [44] train-rmse:254.364661
## [45] train-rmse:247.559291
## [46] train-rmse:241.089479
## [47] train-rmse:234.875905
## [48] train-rmse:228.797457
## [49] train-rmse:223.018926
## [50] train-rmse:217.412544
## [51] train-rmse:211.912976
## [52] train-rmse:206.605702
## [53] train-rmse:201.477395
## [54] train-rmse:196.543392
## [55] train-rmse:191.706514
## [56] train-rmse:187.061459
## [57] train-rmse:182.579076
## [58] train-rmse:178.215604
## [59] train-rmse:174.059760
## [60] train-rmse:170.046247
## [61] train-rmse:166.122830
## [62] train-rmse:162.256133
## [63] train-rmse:158.597422
## [64] train-rmse:154.970564
## [65] train-rmse:151.529335
## [66] train-rmse:148.049284
## [67] train-rmse:144.727509
## [68] train-rmse:141.517196
## [69] train-rmse:138.411421
## [70] train-rmse:135.407500
## [71] train-rmse:132.499863
```

```
## [72] train-rmse:129.657902
## [73] train-rmse:126.929283
## [74] train-rmse:124.316426
## [75] train-rmse:121.747169
## [76] train-rmse:119.258245
## [77] train-rmse:116.797069
## [78] train-rmse:114.453417
## [79] train-rmse:112.195701
## [80] train-rmse:109.998457
## [81] train-rmse:107.830020
## [82] train-rmse:105.756717
## [83] train-rmse:103.699713
## [84] train-rmse:101.606903
## [85] train-rmse:99.711581
## [86] train-rmse:97.814091
## [87] train-rmse:96.026639
## [88] train-rmse:94.180846
## [89] train-rmse:92.504414
## [90] train-rmse:90.820901
## [91] train-rmse:89.118084
## [92] train-rmse:87.594758
## [93] train-rmse:86.047945
## [94] train-rmse:84.489817
## [95] train-rmse:83.019221
## [96] train-rmse:81.573786
## [97] train-rmse:80.195183
## [98] train-rmse:78.838292
## [99] train-rmse:77.427782
## [100]   train-rmse:76.155876
## [101]   train-rmse:74.912054
## [102]   train-rmse:73.715784
## [103]   train-rmse:72.617226
## [104]   train-rmse:71.567143
## [105]   train-rmse:70.519833
## [106]   train-rmse:69.480168
## [107]   train-rmse:68.351614
## [108]   train-rmse:67.328931
## [109]   train-rmse:66.284927
## [110]   train-rmse:65.405866
## [111]   train-rmse:64.441324
## [112]   train-rmse:63.457890
## [113]   train-rmse:62.551651
## [114]   train-rmse:61.639717
## [115]   train-rmse:60.877559
## [116]   train-rmse:60.008401
## [117]   train-rmse:59.127984
## [118]   train-rmse:58.312272
## [119]   train-rmse:57.567018
## [120]   train-rmse:56.623820
## [121]   train-rmse:55.673377
## [122]   train-rmse:54.911253
## [123]   train-rmse:54.013754
## [124]   train-rmse:53.257916
## [125]   train-rmse:52.642787
```

```
## [126]    train-rmse:51.929140
## [127]    train-rmse:51.246517
## [128]    train-rmse:50.695553
## [129]    train-rmse:50.124068
## [130]    train-rmse:49.591109
## [131]    train-rmse:49.032399
## [132]    train-rmse:48.511355
## [133]    train-rmse:47.777392
## [134]    train-rmse:47.124680
## [135]    train-rmse:46.622057
## [136]    train-rmse:46.096792
## [137]    train-rmse:45.622257
## [138]    train-rmse:45.044003
## [139]    train-rmse:44.538095
## [140]    train-rmse:44.113662
## [141]    train-rmse:43.710029
## [142]    train-rmse:43.317176
## [143]    train-rmse:42.949418
## [144]    train-rmse:42.461633
## [145]    train-rmse:42.001810
## [146]    train-rmse:41.572698
## [147]    train-rmse:41.196978
## [148]    train-rmse:40.740245
## [149]    train-rmse:40.338627
## [150]    train-rmse:39.924143
## [151]    train-rmse:39.571284
## [152]    train-rmse:39.100461
## [153]    train-rmse:38.654959
## [154]    train-rmse:38.280954
## [155]    train-rmse:37.984690
## [156]    train-rmse:37.617006
## [157]    train-rmse:37.267770
## [158]    train-rmse:36.830947
## [159]    train-rmse:36.531969
## [160]    train-rmse:36.100781
## [161]    train-rmse:35.777380
## [162]    train-rmse:35.490229
## [163]    train-rmse:35.187396
## [164]    train-rmse:34.916767
## [165]    train-rmse:34.665024
## [166]    train-rmse:34.401656
## [167]    train-rmse:34.013809
## [168]    train-rmse:33.767725
## [169]    train-rmse:33.396265
## [170]    train-rmse:33.166382
## [171]    train-rmse:32.814123
## [172]    train-rmse:32.602863
## [173]    train-rmse:32.269548
## [174]    train-rmse:32.041731
## [175]    train-rmse:31.704151
## [176]    train-rmse:31.381781
## [177]    train-rmse:31.169640
## [178]    train-rmse:30.926821
## [179]    train-rmse:30.584784
```

```
## [180]    train-rmse:30.273622
## [181]    train-rmse:30.120016
## [182]    train-rmse:29.974210
## [183]    train-rmse:29.860189
## [184]    train-rmse:29.708508
## [185]    train-rmse:29.528314
## [186]    train-rmse:29.395991
## [187]    train-rmse:29.211082
## [188]    train-rmse:29.041096
## [189]    train-rmse:28.913385
## [190]    train-rmse:28.648721
## [191]    train-rmse:28.507399
## [192]    train-rmse:28.391300
## [193]    train-rmse:28.268187
## [194]    train-rmse:28.150949
## [195]    train-rmse:28.045894
## [196]    train-rmse:27.967772
## [197]    train-rmse:27.889617
## [198]    train-rmse:27.798920
## [199]    train-rmse:27.645075
## [200]    train-rmse:27.534144
```

Prepare Future Feature Set for 2022

```
X_future <- as.matrix(data.frame(
  lag_1 = tail(y, 1), # Last known case count
  lag_2 = tail(y, 2)[1],
  lag_3 = tail(y, 3)[1],
  lag_7 = tail(y, 7)[1],
  stringency_index = mean(data$stringency_index, na.rm = TRUE) # Assuming policy remains stable
))
```

Predict 365 Days Ahead

```
final_predictions <- numeric(365)
```

Ensure X_future is a matrix with the correct structure

```
X_future <- matrix(c(tail(y, 1), tail(y, 2)[1], tail(y, 3)[1], tail(y, 7)[1],
                    mean(data$stringency_index, na.rm = TRUE)),
                  nrow = 1, byrow = TRUE)

colnames(X_future) <- c("lag_1", "lag_2", "lag_3", "lag_7", "stringency_index")
```

Rolling predictions for 365 days

```
for (i in 1:365) {
  # Convert to xgb.DMatrix
  X_future_dmatrix <- xgb.DMatrix(data = as.matrix(X_future))

  # Make prediction
  final_predictions[i] <- predict(xgb_final, X_future_dmatrix)

  # Create new row with updated lag values
  new_row <- c(final_predictions[i], X_future[1, 1:3], X_future[1, 5])

  # Update X_future (replace oldest row with the new row)
```

```
X_future <- rbind(X_future, new_row)
X_future <- X_future[-1, , drop = FALSE] # Keep it as a matrix
}
```

Print final predictions

```
print(head(final_predictions))
```

```
## [1] 28.0848446 0.2944969 0.2944969 0.2944969 45.1522522 0.2944969
```

Saving the forecast in a CSV file

```
final_results <- data.frame(
  date = seq(max(data$date) + 1, by = "day", length.out = 365),
  predicted_cases = final_predictions
)

write.csv(final_results, "final_predictions_xgboost.csv", row.names = FALSE)
```